

Trei replici, un singur upstream

Exerciții · `compose-scale.yaml` · nginx ca echilibror de trafic

Până acum, fiecare rută ducea la **un** container. Aici o rută duce la mai multe deodată, iar întrebarea nu mai e „ajunge cererea?”, ci „**la care dintre ele, și ce se întâmplă când unul cade?**”

Serviciul `app` nu are `container_name`, tocmai ca să poată fi pornit în mai multe exemplare. `whoami` îți spune în răspuns numele containerului care l-a servit — asta e toată aparatura de măsură de care ai nevoie.

Nu primești `scale.conf`. Îl scrii tu, de trei ori, și de fiecare dată măsoară ce ai obținut.

Fiecare pas cere **numere** în predare, nu impresii.

Pasul 1 — Trei exemplare

```
docker compose -f compose-scale.yaml up -d --scale app=3
```

CRITERIU DE GATA

```
docker compose -f compose-scale.yaml ps
```

`scale-app-1`, `-2`, `-3`, toate *running*, toate cu PORTS gol. Numele încep cu `scale-` pentru că fișierul o cere în prima linie; o să ai nevoie de ele mai târziu.

Pasul 2 — Cea mai simplă configurație posibilă

Un singur `location` / care trimite la serviciul `app`, pe portul 80. Nimic altceva. Apoi măsoară unde ajung cererile:

```
for i in $(seq 30); do
  curl -s http://IP_PUBLIC/ | grep '^Hostname:'
done | sort | uniq -c
```

CRITERIU DE GATA

Apar **toate trei** containerele, aproximativ în părți egale. Dacă vezi unul singur, proxy-ul a pornit înaintea replicilor — e exact lecția din exercițiul precedent. `reload` și măsoară din nou.

Pasul 3 — Adugi replici fără să atingi nginx

```
docker compose -f compose-scale.yaml up -d --scale app=5
```

Măsoară din nou, cu aceeași buclă. Apoi dă `reload` proxy-ului și măsoară a treia oară.

CRITERIU DE GATA

Trei numere în predare: câte containere distincte apar înainte de scalare, după scalare, și după `reload`. Plus o frază care explică de ce al doilea număr e cel pe care l-ai obținut, și nu altul.

Pasul 4 — Cade o replică

```
docker stop scale-app-2
for i in $(seq 100); do
  curl -s -o /dev/null -w '%{http_code}\n' -m 5 http://IP_PUBLIC/
done | sort | uniq -c
```

CRITERIU DE GATA

Numărul exact de cereri care n-au întors `200` . Pornește replica la loc înainte de pasul următor.

Pasul 5 — Se blochează o replică

Altceva decât la pasul 4: containerul rămâne *pornit*, dar nu mai răspunde. Așa arată o aplicație care a rămas agățată, și e mult mai frecvent decât una care moare curat.

```
docker pause scale-app-3
time (for i in $(seq 40); do
  curl -s -o /dev/null -w '%{http_code}\n' -m 4 http://IP_PUBLIC/
done | sort | uniq -c)
docker unpause scale-app-3
```

CRITERIU — două numere, nu unul

Câte cereri au picat **și** cât a durat toată bucla. Al doilea număr e cel interesant: compară-l cu durata aceleiași bucle pe platforma sănătoasă. Apoi explică în două fraze de ce oprit e mai bine decât blocat, din punctul de vedere al lui nginx.

Pasul 6 — Repară

Scrie a treia configurație, astfel încât o replică blocată să nu mai coste nimic. Ai nevoie de trei lucruri: un grup de upstream declarat explicit, limite de timp pentru o cerere către un upstream, și permisiunea ca nginx să încerce alt server când primul nu răspunde. Numele containerelor le ai de la pasul 1.

CRITERIU DE GATA

Refaci exact pasul 5. Toate cele 40 de cereri întorc `200` , iar bucla se termină în mai puțin de 10 secunde. Scrii ambele numere lângă cele de la pasul 5.

Pasul 7 — Prețul

Cu configurația de la pasul 6 pornită, scalează la 6 replici și măsoară din nou distribuția.

CRITERIU DE GATA

Ai acum trei variante încercate: cea de la pasul 2, cea de la pasul 6, și o a treia care rezolvă fix ce s-a stricat acum — o știi din exercițiul precedent. Scrie un tabel cu cele trei pe rânduri și, pe coloane: *vede replici noi?*, *supraviețuiește unei replici oprite?*, *supraviețuiește uneia blocate?* Completează-l cu numerele tale, nu cu păreri. La final, o singură frază: pe ce ai pune serverul tău și de ce.

Ce predai

- cele trei fișiere de configurație, comise în `lectii/L2-server-nginx/upstream-si-scalare/`
- tabelul de la pasul 7, completat cu măsurătorile tale
- răspunsurile în scris la provocări

Când ceva nu merge

Ce vezi	Unde te uiți întâi
toate cererile ajung la același container	nginx a pornit când exista o singură replică. <code>reload</code>
<code>host not found in upstream</code> după ce ai scris numele containerelor	prefixul: containerele se numesc <code>scale-app-N</code> , nu <code>app-N</code>
bucla de la pasul 5 durează zeci de secunde	nu e o defecțiune, e răspunsul la pasul 5. Timpul se limitează, nu se așteaptă
după <code>--scale app=6</code> , două replici nu primesc nimic	le-ai enumerat pe nume la pasul 6. Ți s-a dat prețul
<code>502</code> constant pe toate	<code>docker compose ps</code> — probabil ai uitat <code>unpause</code>

Provocări

1. Serviciul `slow` întârzie la comandă: `/delay/3` răspunde după trei secunde. Rutează-l și pe el, cu mai multe replici, și arată cu măsurători diferența dintre repartizarea implicită și `least_conn`. Când contează diferența și când nu contează deloc?
2. Fă în așa fel încât cererile tale să ajungă mereu la aceeași replică, iar ale altcuiva la alta. Apoi răspunde: ce se strică în distribuție când scalezi de la 3 la 6, și de ce soluția asta e o proteză, nu o rezolvare?
3. `/slow/status/500` întoarce mereu eroare de server. O replică poate fi pornită, rapidă și complet inutilă. nginx **nu** o scoate din rotație pentru asta. Ce ai adăuga — și de ce e periculos să adaugi exact lucrul ăla pe o rută care scrie în baza de date?
4. Între nginx și replici se deschide câte o conexiune nouă la fiecare cerere. Se poate altfel. Adaugă ce trebuie, măsoară aceeași buclă de 100 de cereri înainte și după, și pune cele două durate în predare — plus versiunea de protocol fără de care schimbarea n-are efect.